

One Is Not Enough: The Untold Story of Multiple Security Patches for One Vulnerability

Fangyuan Zhang

College of Computer Science
Nankai University
Tianjin, China
Nanyang Technological University
Singapore
fangyuanzhang@mail.nankai.edu.cn

Lyuye Zhang^{✉*}

College of Cryptology and Cyber
Science
Nankai University
Tianjin, China
Nanyang Technological University
Singapore
zh0004ye@e.ntu.edu.sg

Lingling Fan^{✉*}

DISSec, NDST, College of Cryptology
and Cyber Science
Nankai University
Tianjin, China
linglingfan@nankai.edu.cn

Chengwei Liu

College of Cryptology and Cyber
Science
Nankai University
Tianjin, China
chengwei.liu@nankai.edu.cn

Yinan Li

College of Cryptology and Cyber
Science
Nankai University
Tianjin, China
2120250736@mail.nankai.edu.cn

Liang Huang

Qi An Xin Technology Group
Beijing, China
huangliang@qianxin.com

Yang Liu

Nanyang Technological University
Singapore
yangliu@ntu.edu.sg

Zheli Liu

DISSec, NDST, College of Cryptology
and Cyber Science
Nankai University
Tianjin, China
liuzheli@nankai.edu.cn

Sen Chen

DISSec, NDST, College of Cryptology
and Cyber Science
Nankai University
Tianjin, China
senchen@nankai.edu.cn

Abstract

Security patches (SPs) are the main mechanism for fixing software vulnerabilities, yet a single vulnerability is not always resolved by a single patch: fixes may be completed incrementally, propagated across maintained branches, or replicated across related repositories. When patch records are incomplete, downstream users may observe only part of the required fix set and therefore apply only partial patching. However, comprehensive patch discovery remains difficult because the prevalence and causes of the multi-SP phenomenon are still poorly understood. In this paper, we present the first large-scale empirical study of multi-SP vulnerabilities. By merging four major vulnerability databases, we construct a dataset of 6,053 multi-SP CVEs with 16,260 SPs, showing that 20.6% of CVEs with patches involve multiple SPs and that merging databases increases recognized multi-SP CVE counts by 36-55% over any single source. We further analyze why a vulnerability is associated with multiple SPs and derive a two-level taxonomy with 6 categories and 16 sub-categories. Based on these findings, we develop *SPectre*, a taxonomy-driven prototype for comprehensive patch discovery. On 300 multi-SP CVEs, after manually verifying ground-truth SPs,

SPectre improves multi-SP patch coverage over representative patch-localization baselines, achieving 0.927 recall on same-repository cases and 0.873 recall on cross-repository cases after manual ground-truth verification. On 100 recent CVEs recorded as single-patch by all public databases, *SPectre* further discovers 28 previously unreported SPs across 20 CVEs. Our results show that multi-SP vulnerabilities are both prevalent and systematically underreported, motivating stronger patch-completeness awareness, improved vulnerability database curation, and relation-aware security tooling.

CCS Concepts

• Security and privacy → Software security engineering; • Software and its engineering → Maintaining software.

Keywords

Software Security, Security Patch, Vulnerability Database

ACM Reference Format:

Fangyuan Zhang, Lyuye Zhang, Lingling Fan, Chengwei Liu, Yinan Li, Liang Huang, Yang Liu, Zheli Liu, and Sen Chen. 2026. One Is Not Enough: The Untold Story of Multiple Security Patches for One Vulnerability. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3837455>

1 Introduction

The prevalence of software vulnerabilities has grown exponentially with open-source ecosystems and dependency networks. Each newly disclosed vulnerability can propagate through thousands of

*Lyuye Zhang and Lingling Fan are the corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3837455>

downstream projects [11], amplifying its impact across the software ecosystem. Security patches (SPs), therefore, play a pivotal role in ecosystem-wide mitigation by embedding corrective logic that prevents exploitation and halts further vulnerability diffusion.

Fixing a vulnerability is rarely a one-time job. On the one hand, fixing can often be done incrementally: an initial patch may be incomplete or incorrect, and follow-up commits are needed to address overlooked cases, residual attack surfaces, or newly exposed exploitation vectors. For example, the Log4Shell vulnerability has four successive patches over 18 days before its exploitation vectors were fully addressed [13]. On the other hand, a vulnerability may affect multiple maintained versions or related repositories, and therefore require distinct patches in each of them. For instance, CVE-2022-0778 [14] was patched on several actively maintained OpenSSL branches, each with a different implementation context. In both scenarios, the effective vulnerability fixing is reflected by multiple SPs rather than an isolated commit, also reported by prior studies [44]. When vulnerability databases record only part of that set, downstream users may obtain a partial view of the vulnerability fixing scope. Missing follow-up patches may lead to false belief that a vulnerability has been fully fixed when important attack surfaces remain exposed. Missing branch- or repository-specific patches can prevent maintainers from locating the applicable fix, delay patch deployment, or encourage incorrect patch porting across incompatible contexts. Therefore, improving the coverage of recorded SPs is essential for characterizing real-world vulnerability fixing.

Unfortunately, existing patch localization approaches do not effectively support comprehensive patch discovery for vulnerabilities. Prior studies largely formulate patch identification as commit-level relevance ranking [2, 10, 28, 35, 39] or rely on link tracing across references and branches [28, 49]. While these strategies may rank multiple SPs at high orders, no semantic relationships among SPs are modeled. SHIP [31] moves beyond single-commit identification by predicting pairwise inter-commit relevance to cluster multiple SPs into a group. However, it models inter-commit relations with surface-level statistical signals, counts of shared code entities, cross-reference co-occurrence, and text similarity of commit messages, without reasoning about the deeper semantic dependencies. Existing methods thus remain limited in discovering comprehensive patch sets for multi-SP vulnerabilities. A systematic understanding of why multi-SP vulnerabilities arise and how their SPs are related is therefore essential for improving patch coverage.

To address this gap, we conduct a large-scale empirical study of multi-SP vulnerabilities by merging four vulnerability databases, and develop a prototype tool, named *SPectre*, to operationalize the resulting insights. Specifically, in **RQ1**, we quantify the prevalence of multi-SP vulnerabilities in existing vulnerability databases and measure how cross-source aggregation improves their coverage. In **RQ2**, we investigate why a single vulnerability is associated with multiple SPs by analyzing the semantic relationships among patches, and we construct a taxonomy of the reasons behind the multi-SP phenomenon, comprising 6 categories and 16 sub-categories. In **RQ3**, we translate these findings into *SPectre* and evaluate whether the patterns and taxonomy findings can identify related SPs more effectively than existing patch localization baselines. After manually verifying the ground-truth SPs, *SPectre* achieves 0.927 recall for the same-repository cases, and 0.873 recall

in the cross-repository setting, recovering additional related SPs beyond representative ranking-based baselines at $K=50$. Finally, in **RQ4**, we assess whether *SPectre* can discover previously unreported SPs for recently disclosed CVEs through manual verification and real-world validation, uncovering 28 unreported SPs, one of which has been officially acknowledged by GHSA [3].

In conclusion, we made the following contributions:

- We present the first large-scale empirical study of multi-SP vulnerabilities, integrating four major vulnerability databases, and quantify both their prevalence and the additional coverage obtained through cross-source aggregation.
- We provide a systematic semantic analysis of why a single vulnerability corresponds to multiple SPs, and derive a two-level taxonomy of the reasons behind the multi-SP phenomenon.
- We design *SPectre* to operationalize these insights and demonstrate that it complements existing patch localization baselines by improving coverage of related SPs in multi-SP scenarios.
- We validate *SPectre* on recently disclosed CVEs and uncover 28 previously unreported SPs after manual review, showing that databases systematically under-report multi-SP phenomenon.

2 Background and Research Problem

Definitions. For clarity, we introduce several key concepts used throughout this study:

- **Security Patch (SP):** A code commit that fixes the vulnerability, as often referenced in vulnerability databases [3, 15, 49].
- **Multi-SP Vulnerability:** A disclosed vulnerability explicitly associated with more than one SP in vulnerability databases.

Research Problem. This study aims to systematically investigate the *multi-SP phenomenon*, where multiple SPs correspond to a single vulnerability (CVE), and develop a prototype tool to discover the comprehensive SPs. We seek to understand *how prevalent* multi-SP vulnerabilities are in existing vulnerability databases, *why* a single vulnerability requires multiple SPs, *how* the empirical insights can support the identification of related SPs, and *whether* unreported SPs for CVEs can be discovered in practice.

3 Data Collection Methodology

Although prior work [31] collected a multi-SP dataset, it relies on two vulnerability databases, NVD and Snyk, and may therefore have limited coverage of SPs and multi-SP CVEs. To systematically study the multi-SP phenomenon at a larger scale, we construct *SP_{DB}* by collecting SPs from four widely used and publicly accessible vulnerability databases. Since these databases share a common vulnerability identifier (CVE-ID), we merged the SP records associated with the same CVE across databases with de-duplication.

We leveraged four well-established vulnerability databases used in recent work [10, 47, 48] to obtain SPs with CVEs: NVD [15], Snyk Vulnerability Database [29], GHSA [3], and Google Open Source Vulnerabilities (OSV) database [22]. We crawled CVE entries and associated references from 2005 to 2026, and applied regular expressions to extract links that both contain the keyword “commit” and a commit hash, following the previous works [10, 35].

The detailed data collection methodology is as follows: ① We collected the CVEs with SPs from four major vulnerability databases. ② We merged the extracted SPs from all four vulnerability databases

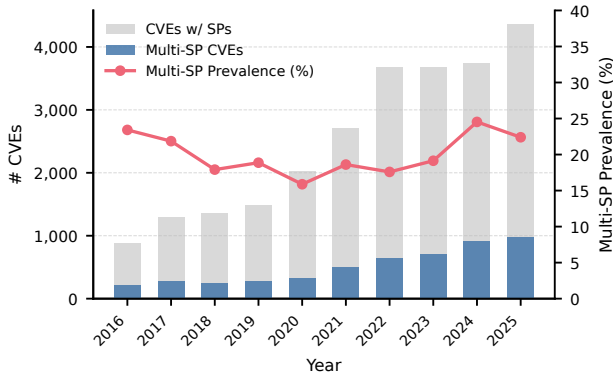


Figure 1: Annual prevalence of multi-SP phenomenon.

on a per-CVE basis with de-duplication. In total, we collected 29,445 CVEs associated with 39,659 SPs. ③ To study the multi-SP vulnerabilities, we filtered out CVEs with one SP and retained only those linked to more than one distinct SP. This process yielded 6,053 CVEs (20.6% of total collected CVEs), associated with 16,260 SPs, constituting a multi-SP dataset as SP_{DB} from the merged SP set of four major vulnerability databases. SP_{DB} serves as the foundation for the subsequent analysis in research questions.

4 Empirical Study

We organize our empirical study around the following RQs:

- RQ1: How prevalent are multi-SP vulnerabilities in existing vulnerability databases?
- RQ2: What are the reasons behind the multi-SP phenomenon for a single vulnerability?
- RQ3: To what extent can our empirical findings support the effective identification of multiple SPs?
- RQ4: Can SPectre find unreported SPs for the recent CVEs?

4.1 RQ1: Prevalence of Multi-SP Vulnerabilities

4.1.1 Prevalence of Multi-SP Vulnerabilities in SP_{DB} . Figures 1 to 3 present three views of multi-SP prevalence in SP_{DB} : the annual prevalence of multi-SP phenomenon, the differences in vulnerability types between all CVEs with SPs and multi-SP CVEs only, and the distribution of patch counts per multi-SP CVE.

Annual trend. As shown in Figure 1, multi-SP CVEs consistently account for 15–25% of all CVEs with security patches throughout 2016–2025, demonstrating that multiple SPs per vulnerability is a systemic and persistent phenomenon. More notably, prevalence grows steadily from 15.9% in 2020 to 24.5% in 2024, suggesting that the multi-SP phenomenon has been intensifying in recent years.

CWE distribution. To examine whether certain vulnerability types are more prone to the multi-SP phenomenon, we compare the top-10 CWE distributions of all CVEs with SPs and of multi-SP CVEs (Figure 2). The two lists share 8 of 10 CWEs. CWE-770 (Allocation of Resources Without Limits or Throttling) and CWE-863 (Incorrect Authorization) enter the multi-SP top-10 but are absent from the general top-10. This may reflect their structural complexity: resource throttling must be enforced at multiple architectural layers

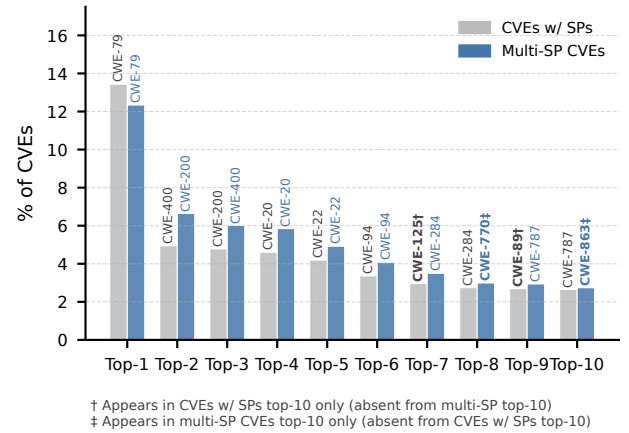


Figure 2: Top-10 CWE distribution in CVEs w/ SPs vs. multi-SP CVEs.

(e.g., API gateway, middleware, and application logic) [53]. Authorization decisions are distributed across modules and are highly context-dependent [30], potentially requiring multiple SPs.

Conversely, CWE-125 (Out-of-bounds Read) and CWE-89 (SQL Injection) are prominent in the general top-10 but absent from the multi-SP top-10. That may be because OOB read bugs tend to be spatially localized [33], and static analysis tools can enumerate the affected buffer access sites, reducing the likelihood of requiring multiple SPs. SQL injection similarly benefits from a well-established fixing pattern (parameterized queries) [6] that can systematically address injection sites within a single refactoring effort.

SP count distribution. Figure 3 shows that 64.4% of multi-SP CVEs in SP_{DB} have exactly two SPs and only 6.7% have five or more, indicating that most cases involve few SPs.

4.1.2 Gaps of Widely Used Vulnerability Databases from SP_{DB} . As shown in Table 1, we compare the number of multi-SP CVEs and average SPs per CVE for each database, standalone vs. after merging all four sources. Merging reveals patches missed by individual databases, increasing both the number of recognized multi-SP CVEs and the average patch count per multi-SP CVE.

NVD benefits the most (+55.1% in CVE coverage, from 2.04 to 2.55 avg. SPs), while GHSA shows the smallest gain (36.1%, 17.9%).

Overall, all four databases exhibit substantial growth after merging, confirming that no single database provides sufficient coverage [5], and integrating multiple sources is essential for constructing SP_{DB} .

Finding (RQ1): Multi-SP vulnerabilities are a systemic and intensifying phenomenon: 20.6% of all CVEs with SPs involve multiple SPs, with annual prevalence rising steadily from 15.9% in 2020 to 24.5% in 2024. Yet this phenomenon is severely underreported: merging all four vulnerability databases increases recognized multi-SP CVE counts by 36–55% over any single source, with NVD showing the largest gain (+55.1%).

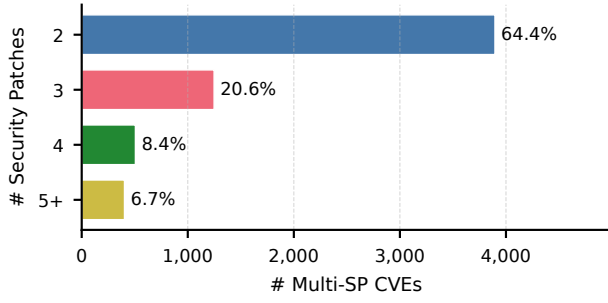


Figure 3: Distribution of SP counts per multi-SP CVE.

Table 1: Multi-SP CVE coverage and average SPs per CVE for each vulnerability database, before and after merging all four databases.

DB	Multi-SP CVE Coverage			Avg. SPs per CVE		
	Own [†]	Merged	$\Delta\%$	Own [†]	Merged	$\Delta\%$
NVD	1,966	3,050	+55.1%	2.04	2.55	+25.1%
Snyk	3,360	4,754	+41.5%	2.15	2.66	+23.7%
GHSA	2,954	4,019	+36.1%	2.33	2.75	+17.9%
OSV	3,610	5,139	+42.4%	2.26	2.72	+20.1%

[†] Using the database alone, without cross-database merging.

4.2 RQ2: Reasons of Multi-SP Phenomenon

4.2.1 Taxonomy Construction. To systematically understand why a single vulnerability is associated with multiple SPs, we analyzed the phenomenon at the patch-pair level. For each CVE, we ordered its SPs by commit date and formed consecutive pairs (sp_1, sp_2). For each pair, we then asked: why is sp_2 necessary to remediate the CVE in the presence of sp_1 ? This pair-centric formulation captures the directional dependency between patches and mirrors the practical scenario in which a developer, already aware of an existing fix, still needs to commit an additional one. Based on these patch pairs, we constructed a two-level taxonomy. The details are as follows.

Dataset. We conducted a dataset, SP_{Tax} , a subset of SP_{DB} , including 4,208 multi-SP CVEs from 2005–2024, for taxonomy construction. The remaining 2025–2026 CVEs in SP_{DB} are reserved as the sampling pool for the tool evaluation in RQ3, reducing the risk of data leakage. For a CVE with n SPs, this yields $n - 1$ patch pairs, resulting in 6,809 patch pairs in total as the unit of analysis.

We constructed the taxonomy through two steps:

(1) *Card Sorting.* Card sorting [32] provides a bottom-up way to surface pair-specific necessity rationales by obtaining the atomic reasons with LLM assistance (GPT-5.2) at scale and manually consolidating them into a hierarchical taxonomy. For each patch pair, the LLM is given the full commit context of both sp_1 and sp_2 , including repository metadata, commit messages, and code diffs. It then follows a two-step chain-of-thought (CoT) protocol. First, it analyzes what each patch addresses and derives a pair-specific necessity rationale explaining why sp_2 remained necessary after sp_1 , treating code diffs as primary evidence and commit metadata as supporting evidence. Second, it compares that rationale against the evolving taxonomy and either assigns the pair to an existing sub-category when appropriate or proposes a new one only when the rationale is genuinely distinct from all existing sub-categories. Applying this

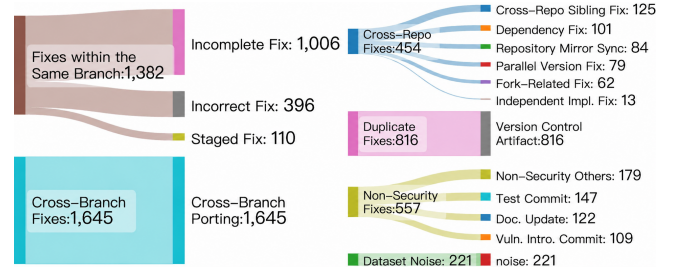


Figure 4: Taxonomy of multi-SP reasons (#CVEs).

process sequentially to all patch pairs produced an initial set of 62 candidate sub-categories. To assess its reliability, two authors with 3+ years of security-analysis experience independently audited a random sample of 50 of the LLM’s pair-level outputs against the underlying commit evidence, finding 94% of them accurate with 96% raw inter-rater agreement (Cohen’s $\kappa=0.73$).

(2) *Taxonomy Consolidation.* These initial sub-categories were intentionally fine-grained to preserve analytical nuance, but they also contained overlaps and redundancies. To organize them into a coherent taxonomy, we first introduced a top-level set of (**categories**) *a priori* to capture the locations of the patch pair at the Git structure, i.e., (1) within a single branch/tag of one repository; (2) parallel patching across multiple branches of the same repository; (3) parallel patching across multiple repositories. During inspection of the initial sub-categories, we also identified three kinds of cases that were not adequately captured by Git structure alone: *duplicate fixes within the same branch*, *non-security fixes*, and *others*. We therefore added them as supplementary categories to ensure full coverage.

After partitioning patch pairs by top-level category, we consolidated the fine-grained categories within each partition into a set of **sub-categories**, following an established taxonomy development method [19]. At this level, each sub-category captures the specific reason why an additional SP was needed within the given patching scope. Two authors independently performed the consolidation by merging, splitting, renaming, or discarding candidate sub-categories based on semantic distinctness, using the guiding criterion that no patch pair should simultaneously satisfy the definitions of two different sub-categories. We iterated until theoretical saturation, i.e., additional patch-pair evidence produced no distinct sub-categories [4], achieving a Cohen’s κ of 0.88 [8]; remaining disagreements were adjudicated by a third expert, yielding the final taxonomy of 6 categories and 16 sub-categories.

4.2.2 Taxonomy Overview. The final taxonomy comprises 6 categories and 16 sub-categories (Figure 4). At the top level, the category captures the roughly coarse-grained scopes of patch pairs. Within each category, the sub-category captures the specific rationale for why an additional SP was still necessary within that scope.

C1: Fixes within the Same Branch. This category covers cases in which multiple SPs for the same CVE occur in the same repository and on the same branch (or tag). Unlike cross-branch or cross-repository cases, the additional patch is required within a single fixing context, meaning that the need for sp_2 arises from how the vulnerability is fixed in that codebase. We identified 3 sub-categories that captured the specific rationale for why sp_2 remained necessary after sp_1 , including incomplete, incorrect, and staged fixes.

```

Vulnerable Code (class.FroxlorInstall.php):
$cmd = $mysqldump . " " . $db
    " -u " . $user
    " --password='" . $pass
    " ' --result-file=" . $file;
exec($cmd);
X unsanitized
X unsanitized
X unsanitized

-----

Patch A (commit Froxlor/Froxlor@62ce21c, incomplete fix):
$cmd = $mysqldump . " " . escapeshellarg($db)
    " -u " . escapeshellarg($user)
    " --password='" . $pass
    " ' --result-file=" . $file;
X unsanitized
✓ fixed
✓ fixed

-----

Patch B (commit Froxlor/Froxlor@7e36127, follow-up):
$cmd = $mysqldump . " " . escapeshellarg($db)
    " -u " . escapeshellarg($user)
    " --password='" . escapeshellarg($pass)
    " ' --result-file=" . $file;
✓ fixed
✓ fixed
✓ fixed

```

Figure 5: An example of an incomplete fix (CVE-2020-10235).

• **Incomplete Fix** (1,006 CVEs, 1,396 patch pairs). *Incomplete Fix* is the dominant same-branch pattern. In this case, sp_1 adopts an appropriate fixing strategy, but leaves part of the attack surface exposed by missing vulnerable locations, execution paths, edge cases, or input variants. This category is especially important because it reflects a *true patching gap*: users and downstream tools may treat sp_1 as the fix even though exploitable behavior remains.

A representative example is CVE-2020-10235 in Froxlor (Figure 5), where the initial patch sanitized only two of three user-controlled arguments in a shell command, leaving the password parameter injectable. A follow-up commit was needed to apply the same `escapeshellarg()` call to the overlooked variable.

• **Incorrect Fix** (396 CVEs, 419 patch pairs). *Incorrect Fix* captures a different failure mode: sp_1 attempts to remediate the vulnerability, but its implementation is itself flawed, for example, due to a logic error, typo, overly restrictive or overly permissive check, functional regression, or a misidentified root cause. Accordingly, sp_2 does not merely extend the protection introduced by sp_1 ; instead, it *modifies*, *reverts*, or *replaces* the faulty security logic.

• **Staged Fix** (110 CVEs, 134 patch pairs). *Staged Fix* covers cases where the patching is decomposed into structurally dependent steps. Here, sp_1 lays the groundwork and sp_2 completes the fix on top of it; the latter could not be implemented without the former. Together, the two commits form a single logical security fix that is deliberately split across commits for implementation reasons.

C2: Cross-Branch Fixes. It covers cases in which the same CVE is remediated across multiple maintained branches of a single repository. Such cases arise naturally in projects that simultaneously maintain LTS, stable, and legacy release lines [12, 34, 51].

• **Cross-Branch Porting** (1,645 CVEs, 2,357 patch pairs). It captures the propagation of an established fix from one branch to another maintained branch in the same repository. Here, the necessity of sp_2 stems from the need to apply the same fix to another supported development line. Depending on branch divergence, sp_2 may appear either as a near-identical cherry-pick or as a substantially adapted re-implementation [23, 46].

Patch A - branch 4. x (commit d541378, minimal fix):

```

// lib/handlebars/helpers/lookup.js
- if (field === 'constructor' && ...) {
+ if (String(field) === 'constructor' && ...) {

```

Patch B - branch 3. x (commit 156061e):

```

... // utils.js: define dangerousPropertyRegex blacklist
// base.js - broader guard replaces constructor-only check
- if (field === 'constructor'
    && !obj.propertyIsEnumerable(field)) {
+ if (dangerousPropertyRegex.test(String(field))
    && !Object.prototype.hasOwnProperty.call(obj, field)) {
... // javascript-compiler.js: add dangerousPropertyRegex guard

```

Figure 6: Cross-branch porting: CVE-2019-20920 (handlebars.js).

An example is CVE-2019-20920 in Handlebars.js (Figure 6). On 4. x, the fix was a one-line `String()` coercion in the lookup helper closing a `toString()` bypass of the constructor check. On 3. x, however, the same vulnerability required a far more extensive patch: blocking hazardous properties and adding `hasOwnProperty` guards in both the runtime and the compiler. This illustrates that the same vulnerability can manifest as substantially different patches depending on the security maturity of each branch.

C3: Cross-Repository Fixes. This category covers cases in which a single vulnerability affects multiple repositories. Unlike cross-branch fixes (C2), C3 concerns vulnerabilities whose patching footprint spans distinct repositories.

• **Sibling/Component Fix** (125 CVEs, 167 patch pairs). As the most common cross-repository pattern, it captures vulnerabilities that manifest across multiple components, plug-ins, or packages in the same ecosystem, each maintained in its own repository.

• **Dependency Fix** (101 CVEs, 106 patch pairs). It captures patches that propagate along an upstream-downstream dependency relationship. sp_1 fixes the vulnerability in an upstream library, while sp_2 brings that patch into a downstream project through a version bump [57, 58], vendored code update, or local port.

• **Repository Mirror Sync** (84 CVEs, 101 patch pairs). It refers to cases where substantially the same codebase is distributed across multiple repositories due to mirroring, monorepo splitting, or read-only sub-repository extraction.

• **Parallel Version Fix** (79 CVEs, 120 patch pairs). It captures projects that maintain different major versions in separate repositories, analogous to cross-branch maintenance, except that the version lines are separated at the repository level.

• **Fork-Related Fix** (62 CVEs, 75 patch pairs). It covers cases where one affected repository is a fork, or otherwise derived from, the other, by the explicit fork relationship.

• **Independent Implementation Fix** (13 CVEs, 13 patch pairs). It covers independent projects that implement the same protocol or specification and thus exhibit the same vulnerability despite no code sharing, fork lineage, or dependency relation.

C4: Duplicate Fixes within the Same Branch. This category captures cases where two SP entries on the same branch correspond to the same underlying fix, with one sometimes subsuming

the other. A sub-category is *Version Control Artifact* (816 CVEs, 1,070 Patch Pairs), where sp_1 and sp_2 are duplicate representations of the same security fix, typically produced by version-control workflows such as merge, rebase, or squash. Although such workflows may span multiple branch histories, they are treated as the same-branch duplicates because their purpose is to update a single release line rather than to introduce a separate fix for another branch or repository.

C5: Non-Security Fixes. These are patch pairs in which at least one linked commit was not itself a security fix, but collateral work surrounding the actual fix. We observed four sub-categories: *Test Commit*, *Vulnerability-Introducing Commit*, *Documentation/Advisory Update*, and *Non-Security Others*, which mainly cover tests, bug-introducing commits, advisory-only changes, and post-fix engineering follow-up. This suggests that CVE-related references often mix the actual fix with surrounding non-security activity.

C6: Dataset Noise (221 CVEs, 235 patch pairs). Unlike C1–C5, which require a reliable intentional or causal relation between the patch pair and the target CVE, C6 captures dataset-noise pairs where at least one commit lacks such a relation, such as unrelated commits, security patches for other CVEs, or commits accidentally linked through shared PRs or releases.

Heterogeneity across Languages and Ecosystems. The prevalence of each category shifts systematically with implementation language and project ecosystem. Cross-branch fixes occur in 57.5% of Python CVEs, against 39.09% across all CVEs, whereas cross-repository fixes occur in only 5.2% (overall 10.79%), a profile of single-repository projects that port across several maintained branches. C and C++ invert this pattern: fixes within the same branch occur in 43.6% and 44.2% of their CVEs (overall 32.84%), and C’s cross-repository share of 20.2% is nearly twice the overall rate, consistent with an ecosystem that lacks a unified package manager and propagates code through vendoring and forks. At the ecosystem level, nearly every vulnerability in projects that maintain several release lines in parallel requires a cross-branch fix (98.2% of Django CVEs, 88.2% of Tomcat CVEs, overall 39.09%). Among cross-repository fixes, the majority (57.3%) involve repositories under the same organization (release-line mirrors, package splits, and plugin-host pairs).

4.2.3 Reliability Validation. To validate the clarity and consistent applicability of our two-level taxonomy, two authors independently classified a stratified random sample of 200 patch pairs using only the sub-category definitions, achieving a Cohen’s κ of 0.95 on the 16 sub-categories. A third author adjudicated the disagreements, yielding a final classification precision of 96.0% (192/200).

Finding (RQ2): We construct a two-level taxonomy of the reasons behind the multi-SP phenomenon, including 6 categories and 16 sub-categories. *Cross-branch fixes* are the dominant reason (1,645 CVEs, 39.09%). *Fixes within the same branch* are the second most common (1,382 CVEs, 32.84%), with *Incomplete Fix* and *Incorrect Fix* being the most prevalent subtypes. *Cross-repository fixes* affect a further 454 CVEs. Besides, a nontrivial portion is attributable to version-control artifacts that duplicate fixes and to non-security commits.

4.3 RQ3: Finding-Driven Prototype Development and Evaluation

4.3.1 Design of SPECTRE. We design *SPECTRE* as a fully automated end-to-end pipeline that, given only a CVE-ID, discovers a comprehensive set of security patches. *SPECTRE* focuses exclusively on *code-level* security fixes, i.e., commits whose diffs modify source files. Commits touching only documentation, tests, build configurations, or dependency lock files are excluded, as they do not constitute actionable security patches for downstream consumers.

As shown in Figure 7, the pipeline consists of three main stages: Stage I for *Seed Intelligence* (discovering initial seed patch commits from public sources) and Stage II for *Multi-Phase Candidate Discovery* (systematically finding candidate commits), followed by Stage III for *LLM-Agent Determination* (determining whether each candidate commit is a true code-level security patch for the target vulnerability). Crucially, the entire pipeline is *taxonomy-driven*: the two-level reason taxonomy established in Section 4.2.1, primarily its three categories (C1: fixes within the same branch, C2: cross-branch fixes, and C3: cross-repository fixes).

Stage I: Seed Intelligence. Given only a CVE-ID, *SPECTRE* automatically discovers seed patch commits (hereafter, *seeds*) by querying multiple public intelligence sources in sequence:

- S1. Vulnerability Database References:** Extract commit URLs from the reference lists of public vulnerability databases (e.g., NVD) and security advisory platforms.
- S2. Pull Request Resolution:** For references that link to pull requests rather than commits, resolve the PR via the GitHub API to obtain merge commits and individual PR commits.
- S3. Repository CVE Grep:** For repositories identified through S1–S2, search the local git clone for commits mentioning the CVE-ID in their message (`git log -grep`).

To ensure that seeds correspond to actionable code-level SPs, we filter out commits whose changed files are entirely non-code, such as documentation, test-only changes, lock files, or empty merge commits, removing noise from advisory references that point to changelog updates or version bumps rather than actual code fixes.

Stage II: Multi-Phase Candidate Discovery. *SPECTRE* applies three discovery phases per seed and merges the resulting candidate commits.

Phase 1: Same-Branch Candidate Discovery targets *C1* (*fixes within the same branch*) scenarios. Within a configurable time window (± 365 days, with a maximum of 2,000 commits) around each seed on the same branch, *SPECTRE* searches for candidate commits. Compared with Prospector’s ± 60 -day search window [28], we adopt a more conservative ± 365 -day window because over 95% of multi-SP CVEs in SP_{DB} have their first and last recorded SP within one year, and cap the scan at 2,000 commits to bound the search cost in highly active projects. *SPECTRE* uses the following candidate-discovery patterns:

- **C1-1: Time proximity with multi-signal support.** Commits that are either in a direct parent-child relationship with the seed, or occur within 7 days of the seed and share at least one corroborating signal, including the same author, overlapping code files, or overlapping modified hunks.

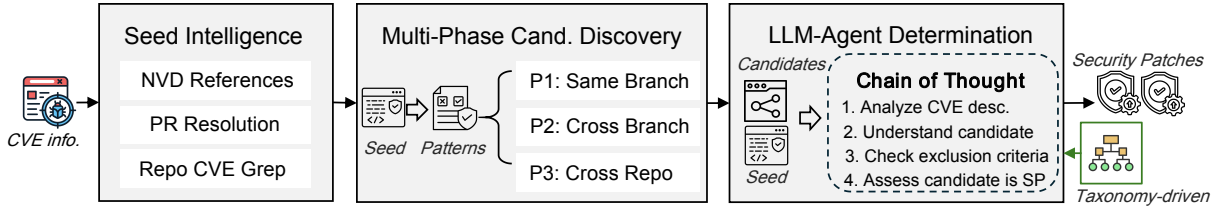


Figure 7: Overview of *SPectre*: a taxonomy-driven prototype for security patch discovery given a CVE ID.

- *C1-2: Identical commit message.* Commits whose normalized message is identical to that of the seed.
- *C1-3/C1-6: CVE-ID/Issue-ID grep.* Commits mentioning the same CVE-ID or issue tracker ID in their message.
- *C1-4: Code evolution.* Commits that last modified the same lines changed by the seed, identified through `git blame` tracing.
- *C1-5: Same pull request.* Commits associated with the same PR as the seed, identified via the GitHub API.
- *C1-7: Commit hash reference.* Commits that reference the seed's commit hash in their message, indicating an explicit relationship.
- *C1-8: Revert Detection.* Commits that revert the seed, which may indicate that the initial fix was later replaced or corrected.
- *C1-9: Fuzzy commit message with multi-signal support.* Commits with a normalized message similar to that of the seed (threshold = 0.65) and at least one additional supporting signal, such as the same author or overlapping code files.

Phase 2: Cross-Branch Candidate Discovery targets *C2* (cross-branch fixes) scenarios. *SPectre* selects branches whose release versions differ from the seed branch, and collects up to 2,000 commits modifying files with the same basenames as those modified by the seed commit, prioritized by temporal proximity to the seed commit. Within this set, we search for candidates using the following candidate-discovery patterns:

- *C2-1: Fuzzy commit message with multi-signal support.* Same as *C1-9*.
- *C2-2: Cherry-pick detection.* Commits reachable from the seed via BFS-based cherry-pick transitive closure, using cherry picked from commit trailers and reverse hash references.
- *C2-3: Overlapping code changes.* Commits whose code hunks are similar with those of the seed, indicating semantically equivalent changes applied independently to separate branches.
- *C2-4/C2-5: Issue-ID/CVE-ID grep.* Same as *C1-6/C1-3*.

Phase 3: Cross-Repository Candidate Discovery targets *C3* (cross-repository fixes) scenarios. Our analysis in Section 4.2 shows that the six sub-categories of *C3* in SP_{Tax} are not distributed arbitrarily across repositories; instead, they typically occur between repositories with some relationships, such as parallel-version repositories or synchronized repositories.

Motivated by this observation, we maintain a repository-relation table that records repository pairs previously observed in SP_{Tax} . Given the repository containing the seed, *SPectre* first queries this table to find potential repositories. If no match is found, *SPectre* falls back to the top-5 starred fork-related repositories returned by the GitHub API. Then we search for candidates within the target repositories using the following candidate-discovery patterns: *C3-1: Identical commit message*, *C3-2: Overlapping code changes*, *C3-3/C3-6: CVE-ID/Issue-ID grep*, *C3-4: Cherry-pick detection*, and *C3-5:*

Identical author date. *SPectre* considers commits whose author-date timestamps exactly match that of the seed's as candidates, since commits derived from the same original fix across repositories often retain it.

Overall, this stage uses candidate-discovery patterns to reduce the raw commit space into a compact candidate pool for subsequent determination. The patterns rely on observable evidence from local Git histories and GitHub metadata, such as repository history, pull-request metadata, commit messages, and local diff/code search, so candidate discovery itself incurs no LLM cost.

Stage III: LLM-Agent Determination. Each candidate identified by the discovery phases above is determined by an LLM-based agent before being added to the output. The agent operates with *taxonomy-informed prompts*: depending on whether the candidate resides on the same branch, a different branch, or a different repository as the seed, one of three specialized system prompts is selected. Each prompt combines exclusion criteria with taxonomy-derived acceptance criteria: a candidate is retained only if it is not excluded and is supported by one sub-category.

The exclusion criteria remove candidates that should not be linked to the input CVE: (1) commits that are not security patches at all, such as tests, documentation, cleanup, version bumps, logging, refactoring, build/CI changes, or vulnerability-introducing commits; and (2) security-related commits that are not clearly tied to the input CVE. For the latter, the agent is explicitly told that temporal proximity, shared authorship, overlapping files, similar paths, or a similar bug class alone are insufficient evidence of the same CVE.

The acceptance criteria are derived from the sub-categories of the corresponding taxonomy partition. For example, a same-branch candidate is accepted only when the evidence linking it to the seed patch supports sub-categories such as incomplete fix, incorrect fix, or staged fix. Thus, a candidate is accepted only if no exclusion criterion applies and there is positive evidence that its relation to the seed patch satisfies one of the taxonomy-derived acceptance criteria for the corresponding scenario.

The agent follows a four-step CoT protocol: (i) analyze the vulnerability from the CVE description, (ii) examine what the candidate commit addresses, (iii) check exclusion criteria, and (iv) match the candidate against the taxonomy-driven acceptance criteria. The agent returns a structured JSON verdict containing the classification (*yes*, *no*, or *uncertain*), and key evidence excerpts from code diffs or commit messages. Only when the evidence is genuinely insufficient (e.g., empty or truncated diffs) does the agent return *uncertain*. Only candidates receiving a *yes* verdict are retained; *no* and *uncertain* verdicts are not included in the output.

4.3.2 Effectiveness Evaluation of *SPectre*. We evaluate *SPectre* as a standalone end-to-end pipeline on a dataset with other baselines: given only a CVE-ID, the tool autonomously discovers seed patch commits from public sources, and expands them into a comprehensive set of related security patches.

Dataset. We construct SP_{Eval} by randomly sampling 300 multi-SP CVEs from the late-2025–2026 portion of SP_{DB} , which keeps it temporally disjoint from SP_{Tax} (2005–2024) to avoid data leakage while remaining feasible for manual verification. SP_{Eval} (2 SPs: 58.7%, 3+ SPs: 41.3%) is broadly consistent with the patch-count distribution of SP_{DB} shown in Figure 3. We then partition the sampled CVEs by their ground-truth patch locations: (1) *Same-repository* (239 CVEs), where all ground-truth SPs reside in a single GitHub repository and cover same-branch or cross-branch scenarios; (2) *Cross-repository* (61 CVEs), where SPs span two or more GitHub repositories. After removing same-branch duplicates and test-/documentation-only commits, we manually validated the remaining 800 SP candidates to filter non-security commits (e.g., vulnerability-introducing commits) and dataset noise defined in our taxonomy (Section 4.2). Two authors independently reviewed the candidates (Cohen’s $\kappa = 0.945$), with disagreements resolved by a third author, yielding 719 ground-truth SPs for 300 CVEs.

Baselines. We selected baselines from the line of work most directly related to ours, i.e., tracing security patches for disclosed vulnerabilities (Section 6.2), covering representative reproducible families: Prospector [28] (heuristic/rule-based tracing), PatchFinder [10] (LLM-based CVE–commit matching), and SHIP [31] (multi-SP-oriented localization). We excluded other recent methods that either lack a public implementation (e.g., PromVPat [56], Taper [27], SPV [37]) or whose released artifact we could not reproduce during our replication attempt (e.g., PatchSeeker [16]). The details are as follows.

- **PatchFinder [10]:** ranks candidate commits using a fine-tuned LLM that scores commits by similarity to CVE descriptions.
- **SHIP [31]:** an LLM-enhanced patch localization approach that explicitly targets multi-SP vulnerabilities, producing a ranked list of groups of candidate commits per repository.
- **Prospector [28]:** applies rule-based heuristics (e.g., advisory references, message keywords) to rank candidates.
- **LLM-Only:** an ablation using the same seed intelligence and candidate pool as *SPectre*, but replacing candidate-discovery patterns and determination with a single GPT-5-mini judgment for each candidate based on the CVE description, seed patch, and candidate commit. This isolates the benefit of *SPectre*’s pattern-based filtering and taxonomy-guided reasoning. Due to its high cost (~108K API calls for 100 CVEs, 67.5× more than *SPectre*), we evaluate it on a 100-CVE subset (80 same-repo + 20 cross-repo).
- **NVD seeds (floor):** establishes the baseline achievable by simply querying NVD references, without any expansion or ranking.

The first three baselines take a CVE-ID and repository URL as input, producing a ranked candidate list, while LLM-Only produces the output with the same format as *SPectre*.

Metrics. We report *recall* (fraction of ground-truth SPs found) and *precision* (fraction of outputs that are true positives). Baselines produce ranked lists; we evaluate at $K=10$ and $K=50$. *SPectre* produces an unordered set capped at 10 (avg. 3.6 patches/CVE). We report the NVD seeds (floor): the recall achievable by simply retrieving commit URLs from NVD references.

Environment. All experiments run on an Ubuntu 24.04 server with 128 CPU cores, 256 GB RAM, and 8 RTX A6000 GPUs. We use the released implementations and default configurations of PatchFinder [10] and Prospector [28]. As SHIP [31] provides no public pre-trained model, we retrain it with the authors’ released scripts and reported configuration (8:1:1 split, lr 1e-4, 20 iterations, batch 48), without tuning on our dataset. *SPectre* uses 20 parallel workers with a 360-second timeout per CVE; LLM determination (Section 4.3.1) uses gpt-5-mini-2025-08-07 [20, 21] with temperature 0.

Manual Verification of Ground Truth. A core premise of our study is that even when existing vulnerability databases do provide SPs for a vulnerability, those patches may still be incomplete. Consequently, SP_{Eval} ’s initial ground truth *GT* cannot be assumed to be complete either: patches that any evaluated baselines discovers but *GT* does not contain are not necessarily false positives; they may be genuine not-recorded SPs. We therefore adopt a two-phase protocol: we first evaluate all tools against the original *GT* on equal footing denoted as *Before manual*. We then perform a balanced ground-truth expansion over the full 300-CVE evaluation set. For *SPectre*, we inspect 516 outputs that passed LLM determination but were absent from *GT*. For each ranking-based baseline (Prospector, PatchFinder, and SHIP), we inspect the top-ranked output for each evaluation query, resulting in 358 candidates from Prospector, 374 from PatchFinder, and 374 from SHIP. Specifically, 315 out-of-*GT* candidates produced by *SPectre* are confirmed as SPs. The inspected outputs of Prospector, PatchFinder, and SHIP contain 202, 28, and 156 confirmed SPs, respectively, although most of these baseline positives are already present in the original *GT*. After merging duplicate confirmations across tools, this manual expansion identifies 335 previously unrecorded SPs, increasing the ground truth to 1,054 SPs in *GT'*, against which all tools are re-evaluated under the *After manual* setting.

Results Before Manual Verification. Table 2 (upper section) shows each tool’s output evaluated against *GT*. On same-repo CVEs, *SPectre* (0.910 recall) discovers more ground-truth SPs than the representative ranking-based baselines, including SHIP at $K=50$ (0.535 recall), while maintaining 0.510 precision. Increasing K from 10 to 50 improves baselines’ recall only marginally while causing their precision to collapse below 4%, indicating that the additional candidates are overwhelmingly noise. On cross-repo CVEs, *SPectre* achieves 0.829 recall, while Prospector at $K=50$ (0.650 recall) with far higher precision. Note that these baselines are designed for single-repository patch discovery. For CVEs with cross-repository scenarios, we ran them separately on each NVD-referenced repository and interleaved the results; they still cannot discover SPs in repositories not already listed by NVD.

Manual Verification Results. Following the manual verification protocol above, we identified **335 previously unrecorded security patches** in SP_{Eval} , including 315 SPs discovered by *SPectre*. As shown in Table 4, most missing SPs are *cross-repository fixes* (206, 61.5%), followed by *fixes within the same branch* (85, 25.4%) and *cross-branch fixes* (44, 13.1%). Among all patterns, C3-1 (Cross-repo (fuzzy message)) is the largest, accounting for 131 missing SPs. These results show that many of *SPectre*’s apparent false positives against *GT* are in fact genuine SPs missed by existing databases.

Table 2: Effectiveness on SP_{Eval} (300 multi-SP CVEs) with ground truth before (abbr. GT) and after (abbr. GT') manual verification, comparing $SPectre$ against three baselines (shown at $K=10$ and $K=50$).

	Tool	K	Same-repo (239)		Cross-repo (61)	
			Recall	Prec.	Recall	Prec.
Before (GT, 719)	NVD seeds (floor)	—	0.149	—	0.179	—
	PatchFinder	10	0.138	0.034	0.057 [†]	0.013 [†]
	SHIP	10	0.509	0.125	0.400 [†]	0.093 [†]
	Prospector	10	0.411	0.101	0.543 [†]	0.127 [†]
	PatchFinder	50	0.363	0.018	0.236 [†]	0.011 [†]
	SHIP	50	0.535	0.026	0.429 [†]	0.020 [†]
	Prospector	50	0.504	0.025	0.650 [†]	0.031 [†]
	<i>SPectre</i>	—	0.910	0.510	0.829	0.298
After (GT', 1054)	NVD seeds (floor)	—	0.107 [↓]	—	0.119 [↓]	—
	PatchFinder	10	0.115 [↓]	0.039	0.061 [†]	0.025 [†]
	SHIP	10	0.391 [↓]	0.134	0.332 [↓]	0.135 [†]
	Prospector	10	0.310 [↓]	0.106	0.385 [↓]	0.157 [†]
	PatchFinder	50	0.289 [↓]	0.020	0.193 [↓]	0.016 [†]
	SHIP	50	0.416 [↓]	0.028	0.369 [↓]	0.030 [†]
	Prospector	50	0.386 [↓]	0.026	0.475 [↓]	0.039 [†]
	<i>SPectre</i>	—	0.927[↑]	0.725[↑]	0.873[↑]	0.548[↑]

[†] Baselines are single-repository tools; for cross-repo CVEs, we run them on each NVD-referenced repository and interleave the results round-robin, so they cannot discover SPs in repositories not already listed by NVD.

Table 3: Cost-effectiveness of $SPectre$ versus the LLM-Only baseline on a random 100-CVE subset of SP_{Eval} , with GT and GT'.

	Tool	Same-repo (80)		Cross-repo (20)	
		Recall	Prec.	Recall	Prec.
GT	LLM-Only	0.821	0.491	0.771	0.180
	<i>SPectre</i>	0.937	0.488	0.812	0.228
GT'	LLM-Only	0.688	0.604	0.663	0.286
	<i>SPectre</i>	0.950	0.726	0.888	0.462

Based on GT', we re-evaluate all tools without changing their outputs (Table 2, lower section). The key observations are:

- **GT correction validates $SPectre$'s output quality.** GT' is built by verifying all four tools' outputs, yet the baselines contribute only 20 of its 335 additions, so their recall decreases as GT' grows beyond what their unchanged outputs cover. $SPectre$ is the only tool whose recall and precision both increase (0.910→0.927 recall, 0.510→0.725 precision on same-repo), confirming that its apparent false positives against GT were in fact genuine SPs missed by databases and other tools.
- **Higher recall with far fewer candidates.** Against GT', $SPectre$ (0.927 same-repo recall, 0.873 cross-repo recall) provides higher multi-SP coverage than every baseline even at $K=50$, while outputting far fewer candidates. This confirms that candidate-discovery patterns (Section 4.3.1) are more effective than generic commit ranking for multi-SP discovery. However, $SPectre$ still misses 90

Table 4: Distribution of 335 ground-truth-missing patches in SP_{Eval} , grouped by candidate-discovery patterns. N/A denotes the SPs contributed only by the baselines, which are not captured by patterns.

Pattern	Description	Count
C1-1	Same-branch (time proximity)	35
C1-4	Same-branch (code evolution)	17
C1-6	Same-branch (CVE-ID grep)	10
C1-2	Same-branch (identical message)	9
C1-3	Same-branch (issue-ID grep)	7
C1-5	Same-branch (same PR)	2
N/A	Same-branch (baseline-only)	5
C2-3	Cross-branch (overlapping code)	24
C2-1	Cross-branch (fuzzy message)	15
N/A	Cross-branch (baseline-only)	5
C3-1	Cross-repo (fuzzy message)	131
C3-2	Cross-repo (cherry-pick)	28
C3-5	Cross-repo (identical author date)	26
C3-4	Cross-repo (issue-ID grep)	7
C3-3	Cross-repo (CVE-ID grep)	4
N/A	Cross-repo (baseline-only)	10
Total		335

SPs in GT'. These FNs are mainly same-repository fixes (66%); the remaining cases (34%) are cross-repository fixes, mostly within the same organization. The dominant cause is candidate-discovery failure: 92% of missed patches never enter the candidate pool, often because the seed patch does not provide enough observable evidence to connect related candidates, such as generic commit messages, overlapping code changes, or explicit repository relations. Only 8% of the missed patches are generated as candidates but rejected during LLM determination.

- **Substantial lift over NVD seeds (floor).** Against GT', $SPectre$ achieves +0.820 same-repo recall and +0.694 cross-repo recall over the NVD seeds (floor), demonstrating that the multi-phase candidate discovery phases search for patches far beyond what public database references provide.
- **LLM-only determination is costly with marginal benefit.** The LLM-Only baseline (Table 3) uses $SPectre$'s seed intelligence and candidate pool as $SPectre$, but omits the pattern-based filtering and the taxonomy-driven determination. On the same 100-CVE subset, $SPectre$ achieves both higher performance for same-repo and cross-repo scenarios, while its candidate-discovery patterns substantially prune the raw search space (up to 2,000 commits per search scope) before LLM determination, resulting in 67.5× fewer LLM calls (~1.6K vs. ~108K). This demonstrates that domain-specific candidate-discovery patterns are both more effective and more efficient than brute-force LLM determination.

Finding (RQ3): On 300 multi-SP CVEs with manually verified ground-truth SPs, $SPectre$ achieves 0.927 same-repo recall and 0.873 cross-repo recall, improving multi-SP patch coverage over all baselines even at $K=50$. Compared with an LLM-only ablation, $SPectre$ achieves higher recall and precision while using 67.5× fewer LLM calls, confirming that candidate-discovery

Table 5: Unreported SP discoveries on 100 CVEs from 2026. *Spectre* finds 28 previously unknown patches across 20 CVEs.

Multi-SP Scenario	Patches	CVEs
Same-branch fixes	25	17
Cross-branch fixes	1	1
Cross-repository fixes	2	2
Total	28	20

patterns are both more effective and more cost-efficient than brute-force LLM determination. Manual verification confirms **335 previously unrecorded SPs** absent from all vulnerability databases, including 315 contributed by *Spectre*.

4.4 RQ4: Discovering Unreported Multiple SPs

This RQ evaluates whether *Spectre* can discover previously unknown SPs for CVEs currently recorded with only a single patch. We select 100 recent CVEs (all from 2026) recorded with only a single patch across all public vulnerability databases, run *Spectre* on each, and manually verify every discovered candidate.

Results: *Spectre* identifies 48 patch candidates across 25 CVEs as potential security patches. Manual review confirms **28 of these 48 candidates (58.3%) as genuine security patches**, spanning **20 CVEs** (the remaining 5 CVEs yielded only false positives). This means that 20% of sampled CVEs have unreported additional patches, which likely underestimates the true prevalence, as follow-up patches for CVEs disclosed in early 2026 may not yet have been released.

As shown in Table 5, the vast majority (89.28%) are fixes within the same branch, i.e., the commits that harden the vulnerability fix but are not tracked by any database. This implies that applying only the database-recorded patch often leaves residual attack surface unaddressed, with practical consequences for organizations relying on NVD or GHSA for patch management.

Finding (RQ4): Among 100 recent CVEs, *Spectre* discovers **28 previously unreported security patches** across 20 CVEs (20%), demonstrating that vulnerability databases systematically under-report multi-SP phenomenon.

5 Discussion

5.1 Threats to Validity

- **Dataset Construction:** SP_{DB} is merged from four databases, so it may include incorrect SPs and omit others. Since no public dataset provides complete and accurate SPs, we mitigate this by drawing only on widely used, peer-reviewed sources.

- **Reliance on commits only:** Since SPs usually appear as commits in OSS repositories [35], we only focused on the commit links in the references from vulnerability databases. We did not rely on the Patch label offered by some databases, as it is neither always available.

- **Reliance on GitHub only:** Given that extracting commit information from multiple platforms (e.g., GitHub, and Bitbucket) requires maintaining platform-specific API interfaces, we focus on SPs hosted on GitHub. Based on all multi-SP patch links we

collected from 2005–2026, only 3.3% of CVEs have patches hosted exclusively outside GitHub, spanning 93 repositories on platforms such as git.kernel.org, gitlab.com, and bitbucket.org. These cases still cover the main taxonomy categories, so the exclusion is unlikely to remove an entire category of multi-SP behavior. Because linking practices may differ across platforms, however, we cannot rule out platform-specific distribution shifts.

- **LLM-based Labeling:** Bias of llm-based labeling for analyzing reasons in Section 4.2 could be inevitable. To mitigate this threat, we adopted a cross-review process and further validated a stratified random sample to confirm the reliability of the labeling, as described in Section 4.2.3. These measures improve the accuracy of commit annotation and strengthen the reliability of our conclusions.

- **Limitations of discovering SPs with weak observable patterns:** *Spectre*’s candidate-discovery stage requires observable evidence to bring a commit into the candidate pool before LLM determination, such as shared issue/PR/CVE references. Therefore, SPs that lack such observable candidate-discovery patterns can be missed. However, we found that 3.0% of patch pairs in SP_{DB} lack strong observable patterns, suggesting that these cases exist but are relatively uncommon.

- **Reliance on observed repository relations:** *Spectre*’s cross-repository phase uses a relation table built from SP_{Tax} to efficiently handle recurring repository relations, but may miss repositories with no previously observed relation. To mitigate this, *Spectre* also falls back to fork-related repositories, and we found that disabling the table affected only 7/61 CVEs and reduced recall and precision by 5.5 and 1.2 pp, respectively, suggesting that the cross-repository results are not primarily driven by memorized repository pairs.

5.2 Lessons Learned

Our empirical analysis of 6,053 multi-SP CVEs across major vulnerability databases reveals several key takeaways that advance the understanding and management of SPs.

5.2.1 Lesson 1: Missing Recorded Patches for Multi-SP CVEs Are Dangerous. For *Incomplete Fix* and *Incorrect Fix* as shown in Figure 4, an unlinked corrective patch leaves downstream maintainers and tools assessing, synchronizing, or porting a fix that is already known to be inadequate [55, 59]. In *Cross-Branch Porting*, and cross-repository categories such as *Dependency Fix*, an unlinked patch hides the branch- or repository-specific fix needed in another maintenance context. Partial patch records therefore obscure the full fixing scope, delay correct patch deployment, and can lead downstream users or tools to miss the patch that is actually applicable to their software.

5.2.2 Lesson 2: Effective Multi-SP Tracing Needs Relation-Aware and Cross-Source Analysis. Our tool evaluation (Section 4.3.2) shows that the relevance between CVE descriptions and commits is not sufficient on its own but accounting for the relations among SPs across time, branches, and repositories can improve tracing effectiveness. This suggests that future patch localization tools should leverage cross-source evidence and relation-aware signals such as temporal proximity, commit message similarity, and overlapping code changes to discover more comprehensive SPs.

5.2.3 Lesson 3: Directions for Community and Ecosystem Enhancement. Section 4.1.1 shows that no single vulnerability database provides comprehensive coverage of multi-SP CVEs. Thus, the community needs stronger coordination and standardized metadata for patch relationships. Databases should label incomplete, incorrect, follow-up, branch-specific, and repository-specific patches to improve traceability. Security practitioners and database maintainers can improve transparency and patch coverage by explicitly linking related commits and documenting how a vulnerability is fixed across branches or repositories. Ranking-based tools should also balance the recall gains of larger top- K values against their added manual validation cost.

6 Related Work

6.1 Empirical Study of Patch Management

Previous research has examined diverse aspects of security patching in open-source software ecosystems. Li et al. [9] analyzed patch size, latency, and regression risks at scale. Tan et al. [34] studied security-patch propagation across software branches. Ramkisoen et al. [26] investigated missed and duplicated patches in forked projects. Dissanayake et al. [1] examined automation in patch management through practitioner interviews. Xie et al. [48] studied post-deployment patch evolution and its impact on vulnerability-analysis tools. Woo et al. [44] assessed NVD patch effectiveness and found many incomplete or unreliable patches. Park et al. [24, 25] studied multi-fix bugs in general defect repair, finding that about one-quarter of bug reports require supplementary fixes, consistent with our observations on security patches.

These studies cover patch latency, propagation, effectiveness, automation, and fork coordination, but they do not systematically study the multi-SP phenomenon itself, including its prevalence and underlying causes. Our work addresses this gap and uses the resulting taxonomy to guide *SPectre* in discovering comprehensive SP sets for a given CVE.

6.2 Tracing Security Patches for Disclosed Vulnerabilities

A large body of prior work studies how to identify SPs for a disclosed vulnerability, typically formulating the task as ranking candidate commits by their relevance to a CVE description. Early methods rely on handcrafted textual, structural, or link-based signals, including PatchScout [35], VCMATCH [39], VFCFinder [2], Prospector [28], and Tracer [49].

More recent studies strengthen ranking with pretrained models, LLMs, and contextual constraints. PromVPat [56], PatchFinder [10], PatchSeeker [16], Taper [27], and Xu et al. [50] improve CVE-commit semantic matching with PLMs/LLMs, while other hybrid frameworks incorporate vulnerability metadata, repository context, and temporal cues [7, 17].

Several studies go beyond identifying a single patch. SPV [37] applies rule-based analysis to locate branch-level or backported variants once a reference SP is known. SHIP [31] is the closest to our research, as it explicitly considers the scenarios in which one CVE can correspond to multiple SPs. However, SHIP does not explicitly investigate the multi-SP phenomenon itself, nor does it distinguish the different scenarios under which multiple SPs arise

for the same vulnerability. Instead, it treats all related commits in a largely uniform manner. Therefore, the relevance of two commits is still modeled mainly through surface-level similarities, such as shared code entities and text similarity, and the final identification is ultimately resolved through ranking.

Overall, existing ranking-based approaches are not well-suited to recovering comprehensive SPs for a multi-SP CVE, because there is no principled top- K cutoff that guarantees patch coverage. In contrast, our work constructs a two-level taxonomy of the multi-SP phenomenon and uses it to guide *SPectre*: candidate-discovery patterns bound the search space without a top- K cutoff, and LLM-Agent determination judges whether each candidate is a genuine SP for the target vulnerability.

6.3 Identifying Silent Security Patches

Prior work has also explored the identification of silent security patches, those fixing vulnerabilities that have not yet been publicly disclosed or linked to a CVE ID.

Early studies [18, 42, 45, 61, 62] leveraged deep neural networks to learn from commit messages, code differences, or GitHub issues for automatic security patch detection, while Wang et al. [40, 41] further characterized the prevalence and security implications of such secret security patches. Subsequent efforts advanced this line with richer code representations: GraphSPD [38] and CoLeFunDa [60] modeled code changes as graphs to capture syntactic and semantic dependencies, and Wen et al. [43] further scaled this idea to the repository level to account for cross-file dependencies. More recently, Tang et al. [36] and Yang et al. [52] incorporated LLMs into this task: the former augments patch representations with LLM-generated code change explanations and contrastive learning, whereas the latter enriches commit representations with development artifacts, historical vulnerability fixes, and code changes.

These studies address a fundamentally different research problem: they aim to classify arbitrary commits as security fixes in the absence of any CVE linkage, whereas our work investigates the multi-SP phenomenon for disclosed CVEs.

7 Conclusion

We conduct the first large-scale empirical study of the multi-SP phenomenon, revealing that multiple SPs for a single vulnerability are prevalent. By constructing a two-level taxonomy (including 6 categories and 16 sub-categories) of reasons behind this phenomenon, we ground the design of *SPectre*, a taxonomy-driven prototype that recovers additional SPs beyond existing patch localization tools and uncovers 28 previously unreported SPs in recently disclosed CVEs. Our findings call for greater awareness of multi-SP vulnerabilities among database maintainers, tool builders, and security practitioners alike.

Acknowledgments

This work was supported by the National Key R&D Program of China (Grant No. 2024YFE0203800), the Beijing-Tianjin-Hebei Natural Science Foundation Cooperation Project (Grant No. 25JJJC0003), the Tianjin Major Science and Technology Special Project (Grant No. 25ZXSF000140), and the China Scholarship Council (Grant No. 202506200059).

Data Availability Statement

The dataset and scripts of this study are publicly available at [54].

References

- [1] Nesara Dissanayake, Asangi Jayatilaka, Mansoor Zahedi, and Muhammad Ali Babar. 2022. An empirical study of automation in software security patch management. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13. doi:10.1145/3551349.3556969
- [2] Trevor Dunlap, Elizabeth Lin, William Enck, and Bradley Reaves. 2024. VFCFinder: Pairing Security Advisories and Patches. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1128–1142. doi:10.1145/3634737.3657007
- [3] GitHub Advisory Database. 2025. <https://github.com/advisories>.
- [4] Barney Glaser and Anselm Strauss. 2017. *Discovery of grounded theory: Strategies for qualitative research*. Routledge. doi:10.4324/9780203793206
- [5] Hao Guo, Sen Chen, Zhenchang Xing, Xiaohong Li, Yude Bai, and Jiamou Sun. 2022. Detecting and augmenting missing key aspects in vulnerability descriptions. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 3 (2022), 1–27. doi:10.1145/3498537
- [6] William GJ Halfond, Jeremy Viegas, Alessandro Orso, et al. 2006. A classification of SQL injection attacks and countermeasures. (2006).
- [7] Daan Hommersom, Antonino Sabetta, Bonaventura Coppola, Dario Di Nucci, and Damian A Tamburri. 2024. Automated mapping of vulnerability advisories onto their fix commits in open source repositories. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–28. doi:10.1145/3649590
- [8] J Richard Landis and Gary G Koch. 1977. The measurement of observer agreement for categorical data. *biometrics* (1977), 159–174. doi:10.2307/2529310
- [9] Frank Li and Vern Paxson. 2017. A large-scale empirical study of security patches. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2201–2215. doi:10.1145/3133956.3134072
- [10] Kaixuan Li, Jian Zhang, Sen Chen, Han Liu, Yang Liu, and Yixiang Chen. 2024. PatchFinder: A two-phase approach to security patch tracing for disclosed vulnerabilities in open-source software. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 590–602. doi:10.1145/3650212.3680305
- [11] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th international conference on software engineering*. 672–684. doi:10.1145/3510003.3510142
- [12] Rongkai Liu, Heyuan Shi, Shuning Liu, Chao Hu, Sisheng Li, Yuheng Shen, Runzhe Wang, Xiaohai Shi, and Yu Jiang. 2025. PatchScope: LLM-Enhanced Fine-Grained Stable Patch Classification for Linux Kernel. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1513–1535. doi:10.1145/3728944
- [13] Log4j CVE. 2021. <https://nvd.nist.gov/vuln/detail/cve-2021-44228>.
- [14] MITRE Corporation. 2022. CVE-2022-0778. <https://nvd.nist.gov/vuln/detail/cve-2022-0778>. Accessed: 2026-03-10.
- [15] National Vulnerability Database. 2025. <https://nvd.nist.gov/>.
- [16] Huu Hung Nguyen, Anh Tuan Nguyen, Thanh Le-Cong, Yikun Li, Han Wei Ang, Yide Yin, Frank Liauw, Shar Lwin Khin, Ouh Eng Lieh, Ting Zhang, et al. 2025. PatchSeeker: Mapping NVD Records to their Vulnerability-fixing Commits with LLM Generated Commits and Embeddings. *arXiv preprint arXiv:2509.07540* (2025). doi:10.48550/arXiv.2509.07540
- [17] Huu Hung Nguyen, Ting Zhang, Duc Manh Tran, Yiran Cheng, Thanh Le-Cong, Hong Jin Kang, Ratnadira Widayarsi, Shar Lwin Khin, Ouh Eng Lieh, and David Lo. 2026. Mapping NVD Records to Their Vulnerability-fixing Commits: How Hard is It? *ACM Trans. Softw. Eng. Methodol.* (May 2026). doi:10.1145/3817046 Just Accepted.
- [18] Truong Giang Nguyen, Thanh Le-Cong, Hong Jin Kang, Xuan-Bach D Le, and David Lo. 2022. Vulcurator: a vulnerability-fixing commit detector. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1726–1730. doi:10.1145/3540250.3558936
- [19] Robert C Nickerson, Upkar Varshney, and Jan Muntermann. 2013. A method for taxonomy development and its application in information systems. *European journal of information systems* 22, 3 (2013), 336–359. doi:10.1057/ejis.2012.26
- [20] OpenAI. 2025. Introducing GPT-5 for Developers. <https://openai.com/index/introducing-gpt-5-for-developers/>. Accessed: 2026-07-16.
- [21] OpenRouter. 2025. OpenAI: GPT-5 Mini. <https://openrouter.ai/openai/gpt-5-mini>. Accessed: 2026-07-16.
- [22] OSV - Open Source Vulnerabilities. 2025. <https://osv.dev/>.
- [23] Shengyi Pan, You Wang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2024. Automating zero-shot patch porting for hard forks. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 363–375. doi:10.1145/3650212.3652134
- [24] Jihun Park, Miryung Kim, and Doo-Hwan Bae. 2017. An empirical study of supplementary patches in open source projects. *Empirical Software Engineering* 22, 1 (2017), 436–473. doi:10.1007/s10664-016-9432-x
- [25] Jihun Park, Miryung Kim, Baishakhi Ray, and Doo-Hwan Bae. 2012. An empirical study of supplementary bug fixes. In *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*. IEEE, 40–49. doi:10.1109/MSR.2012.6224298
- [26] Poedjavekie Kadjel Ramkisoen, John Businge, Brent Van Bladel, Alexandre Decan, Serge Demeyer, Coen De Roover, and Foutse Khomh. 2022. PaReco: patched clones and missed patches among the divergent variants of a software family. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 646–658. doi:10.1145/3540250.3549112
- [27] Dezhi Ran, Lin Li, Liuchuan Zhu, Yuan Cao, Landelong Zhao, Xin Tan, Guangtai Liang, Qianxiang Wang, and Tao Xie. 2025. Efficient and Robust Security-Patch Localization for Disclosed OSS Vulnerabilities with Fine-Tuned LLMs in an Industrial Setting. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 262–273. doi:10.1145/3696630.3728551
- [28] Antonino Sabetta, Serena Elisa Ponta, Rocio Cabrera Lozoya, Michele Bezzi, Tommaso Sacchetti, Matteo Greco, Gergő Balogh, Péter Hegedűs, Rudolf Ferenc, Ranindya Paramitha, et al. 2024. Known vulnerabilities of open source projects: Where are the fixes? *IEEE Security & Privacy* 22, 2 (2024), 49–59. doi:10.1109/MSEC.2023.3343836
- [29] Snyk Vulnerability Database. 2025. <https://security.snyk.io/>.
- [30] Soeul Son, Kathryn S McKinley, and Vitaly Shmatikov. 2013. Fix Me Up: Repairing Access-Control Bugs in Web Applications. In *NDSS*.
- [31] Yi Song, Dongchen Xie, Lin Xu, He Zhang, Chunying Zhou, and Xiaoyuan Xie. 2025. Not Every Patch is an Island: LLM-Enhanced Identification of Multiple Vulnerability Patches. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 996–1007. doi:10.1109/ASE63991.2025.00087
- [32] Donna Spencer. 2009. *Card sorting: Designing usable categories*. Rosenfeld Media.
- [33] Laszlo Szekeres, Mathias Payer, Tao Wei, and Dawn Song. 2013. Sok: Eternal war in memory. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 48–62. doi:10.1109/SP.2013.13
- [34] Xin Tan, Yuan Zhang, Jiajun Cao, Kun Sun, Mi Zhang, and Min Yang. 2022. Understanding the Practice of Security Patch Management across Multiple Branches in OSS Projects. In *Proceedings of the ACM Web Conference 2022* (Virtual Event, Lyon, France) (WWW '22). Association for Computing Machinery, New York, NY, USA, 767–777. doi:10.1145/3485447.3512236
- [35] Xin Tan, Yuan Zhang, Chenyuan Mi, Jiajun Cao, Kun Sun, Yifan Lin, and Min Yang. 2021. Locating the security patches for disclosed oss vulnerabilities with vulnerability-commit correlation ranking. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3282–3299. doi:10.1145/3460120.3484593
- [36] Xunzhu Tang, Kisub Kim, Saad Ezzini, Yewei Song, Haoye Tian, Jacques Klein, and Tegawede Bissyande. 2025. Just-in-time detection of silent security patches. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3749370
- [37] Lin Wang, Yuan Zhang, Xiaoting Chen, and Min Yang. 2025. Locating Security Patch Variants with Two-Dimensional Code Commit Features. *IEEE Transactions on Information Forensics and Security* (2025). doi:10.1109/TIFS.2025.3577429
- [38] Shu Wang, Xinda Wang, Kun Sun, Sushil Jajodia, Haining Wang, and Qi Li. 2023. Graphspd: Graph-based security patch detection with enriched code semantics. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2409–2426. doi:10.1109/SP46215.2023.10179479
- [39] Shichao Wang, Yun Zhang, Liangfeng Bao, Xin Xia, and Minghui Wu. 2022. Vc-match: a ranking-based approach for automatic security patches localization for OSS vulnerabilities. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 589–600. doi:10.1109/SANER53432.2022.00076
- [40] Xinda Wang, Kun Sun, Archer Batcheller, and Sushil Jajodia. 2019. Detecting “0-day” vulnerability: An empirical study of secret security patch in OSS. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 485–492. doi:10.1109/DSN.2019.00056
- [41] Xinda Wang, Kun Sun, Archer Batcheller, and Sushil Jajodia. 2020. An empirical study of secret security patch in open source software. *Adaptive Autonomous Secure Cyber Systems* (2020), 269–289. doi:10.1007/978-3-030-33432-1_13
- [42] Xinda Wang, Shu Wang, Pengbin Feng, Kun Sun, Sushil Jajodia, Sanae Benchaaboun, and Frank Geck. 2021. Patchrnn: A deep learning-based system for security patch identification. In *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)*. IEEE, 595–600. doi:10.1109/MILCOM52596.2021.9652940
- [43] Xin-Cheng Wen, Zirui Lin, Cuiyun Gao, Hongyu Zhang, Yong Wang, and Qing Liao. 2025. Repository-level graph representation learning for enhanced security patch detection. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1–13. doi:10.1109/ICSE55347.2025.00121
- [44] Seunghoon Woo, Eunjin Choi, and Heejo Lee. 2025. A large-scale analysis of the effectiveness of publicly reported security patches. *Computers & Security* 148 (2025), 104–181. doi:10.1016/j.cose.2024.104181
- [45] Bozhi Wu, Shangqing Liu, Ruitao Feng, Xiaofei Xie, Jingkai Siow, and Shang-Wei Lin. 2022. Enhancing security patch identification by capturing structures in commits. *IEEE Transactions on Dependable and Secure Computing* (2022).

- doi:10.1109/TDSC.2022.3192631
- [46] Susheng Wu, Ruisi Wang, Yiheng Cao, Bihuan Chen, Zhuotong Zhou, Yiheng Huang, JunPeng Zhao, and Xin Peng. 2025. Mystique: Automated Vulnerability Patch Porting with Semantic and Syntactic-Enhanced LLM. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 130–152. doi:10.1145/3715718
 - [47] Susheng Wu, Ruisi Wang, Kaifeng Huang, Yiheng Cao, Wenyan Song, Zhuotong Zhou, Yiheng Huang, Bihuan Chen, and Xin Peng. 2024. Vision: Identifying affected library versions for open source software vulnerabilities. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1447–1459. doi:10.1145/3691620.3695516
 - [48] Zifan Xie, Ming Wen, Zichao Wei, and Hai Jin. 2024. Unveiling the Characteristics and Impact of Security Patch Evolution. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1094–1106. doi:10.1145/3691620.3695488
 - [49] Congying Xu, Bihuan Chen, Chenhao Lu, Kaifeng Huang, Xin Peng, and Yang Liu. 2022. Tracking patches for open source software vulnerabilities. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 860–871. doi:10.1145/3540250.3549125
 - [50] Haoran Xu, Chen Zhi, Junxiao Han, Xinkui Zhao, Jianwei Yin, and Shuguang Deng. 2025. Revisiting Vulnerability Patch Localization: An Empirical Study and LLM-Based Solution. *arXiv preprint arXiv:2509.15777* (2025). doi:10.48550/arXiv.2509.15777
 - [51] Su Yang, Yang Xiao, Zhengzi Xu, Chengyi Sun, Chen Ji, and Yuqing Zhang. 2023. Enhancing oss patch backporting with semantics. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2366–2380. doi:10.1145/3576915.3623188
 - [52] Xu Yang, Wenhan Zhu, Michael Pacheco, Jiayuan Zhou, Shaowei Wang, Xing Hu, and Kui Liu. 2025. Code change intention, development artifact, and history vulnerability: Putting them together for vulnerability fix detection by llm. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 489–510. doi:10.1145/3715738
 - [53] Saman Taghavi Zargar, James Joshi, and David Tipper. 2013. A survey of defense mechanisms against distributed denial of service (DDoS) flooding attacks. *IEEE communications surveys & tutorials* 15, 4 (2013), 2046–2069. doi:10.1109/SURV.2013.031413.00127
 - [54] Fangyuan Zhang. 2026. One Is Not Enough: The Untold Story of Multiple Security Patches for One Vulnerability. doi:10.5281/zenodo.21410721
 - [55] Fangyuan Zhang, Lingling Fan, Sen Chen, Miaoying Cai, Sihan Xu, and Lida Zhao. 2024. Does the vulnerability threaten our projects? Automated vulnerable API detection for third-party libraries. *IEEE Transactions on Software Engineering* 50, 11 (2024), 2906–2920. doi:10.1109/TSE.2024.3454960
 - [56] Junwei Zhang, Xing Hu, Lingfeng Bao, Xin Xia, and Shanping Li. 2024. Dual Prompt-Based Few-Shot Learning for Automated Vulnerability Patch Localization. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 940–951. doi:10.1109/SANER60148.2024.00102
 - [57] Lyuye Zhang, Chengwei Liu, Sen Chen, Zhengzi Xu, Lingling Fan, Lida Zhao, Yiran Zhang, and Yang Liu. 2023. Mitigating persistence of open-source vulnerabilities in maven ecosystem. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 191–203. doi:10.1109/ASE56229.2023.00058
 - [58] Lyuye Zhang, Chengwei Liu, Zhengzi Xu, Sen Chen, Lingling Fan, Lida Zhao, Jiahui Wu, and Yang Liu. 2023. Compatible remediation on vulnerabilities from third-party libraries for java projects. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2540–2552. doi:10.1109/ICSE48619.2023.00212
 - [59] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972. doi:10.1145/3611643.3616299
 - [60] Jiayuan Zhou, Michael Pacheco, Jinfu Chen, Xing Hu, Xin Xia, David Lo, and Ahmed E Hassan. 2023. Colefunda: Explainable silent vulnerability fix identification. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2565–2577. doi:10.1109/ICSE48619.2023.00214
 - [61] Jiayuan Zhou, Michael Pacheco, Zhiyuan Wan, Xin Xia, David Lo, Yuan Wang, and Ahmed E. Hassan. 2022. Finding a needle in a haystack: automated mining of silent vulnerability fixes. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (Melbourne, Australia) (ASE '21)*. IEEE Press, 705–716. doi:10.1109/ASE51524.2021.9678720
 - [62] Yaqin Zhou, Jing Kai Siow, Chenyu Wang, Shangqing Liu, and Yang Liu. 2021. Spi: Automated identification of security patches via commits. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–27. doi:10.1145/3468854

Received 2026-03-26; accepted 2026-06-18